

# Research Notes on Backpropagation

## Backpropagation

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

### >> Section 1

Early successes Contributing to the acceptance were several applications in training neural networks via backpropagation, sometimes achieving popularity outside the research circles. In 1987, NETtalk learned to convert English text into pronunciation. Sejnowski tried training it with both backpropagation and Boltzmann machine, but found the backpropagation significantly faster, so he used it for the final NETtalk. The NETtalk program became a popular success, appearing on the Today show. In 1989, Dean A. Pomerleau published ALVINN, a neural network trained to drive autonomously using backpropagation. The LeNet was published in 1989 to recognize handwritten zip codes. In 1992, TD-Gammon achieved top human level play in backgammon. It was a reinforcement learning agent with a neural network with two layers, trained by backpropagation. In 1993, Eric Wan won an international pattern recognition contest through backpropagation.

### >> Section 2

Using a Hessian matrix of second-order derivatives of the error function, the Levenberg–Marquardt algorithm often converges faster than first-order gradient descent, especially when the topology of the error function is complicated. It may also find solutions in smaller node counts for which other methods might not converge. The Hessian can be approximated by the Fisher information matrix. As an example, consider a simple feedforward network. At the  $l$ -th layer, we have  $x_i^{(l)}$ ,  $a_i^{(l)} = f(x_i^{(l)})$ ,  $x_i^{(l+1)} = \sum_j W_{ij} a_j^{(l)}$   $\{ \displaystyle x_i^{(l)}, \quad a_i^{(l)} = f(x_i^{(l)}), \quad x_i^{(l+1)} = \sum_j W_{ij} a_j^{(l)} \}$  where  $x$  are the pre-activations,  $a$  are the activations, and  $W$  is the weight matrix. Given a loss function  $L$ , the first-order backpropagation states that  $\frac{\partial L}{\partial a_j^{(l)}} = \sum_i W_{ij} \frac{\partial L}{\partial x_i^{(l+1)}}$ ,  $\frac{\partial L}{\partial x_j^{(l)}} = f'(x_j^{(l)}) \frac{\partial L}{\partial a_j^{(l)}}$   $\{ \displaystyle \frac{\partial L}{\partial a_j^{(l)}} = \sum_i W_{ij} \frac{\partial L}{\partial x_i^{(l+1)}}, \quad \frac{\partial L}{\partial x_j^{(l)}} = f'(x_j^{(l)}) \frac{\partial L}{\partial a_j^{(l)}} \}$  and the second-order backpropagation states that  $\frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} = \sum_{i,j} W_{ij} \frac{\partial^2 L}{\partial x_i^{(l+1)} \partial x_j^{(l+1)}} \frac{\partial x_i^{(l+1)}}{\partial a_j^{(l)}} \frac{\partial x_j^{(l+1)}}{\partial a_k^{(l)}} + f''(x_j^{(l)}) \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_k^{(l)}}{\partial a_j^{(l)}}$

$\frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} = f''(x_j^{(l)}) f'(x_k^{(l)}) \frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} + \delta_j \frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} \{ \displaystyle \frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} = f''(x_j^{(l)}) f'(x_k^{(l)}) \frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} + \delta_j \frac{\partial^2 L}{\partial a_j^{(l)} \partial a_k^{(l)}} \}$  where  $\delta$  is the Dirac delta symbol. Arbitrary-order derivatives in arbitrary computational graphs can be computed with backpropagation, but with more complex expressions for higher orders.

### >> Section 3

This is the reason why backpropagation requires that the activation function be differentiable. (Nevertheless, the ReLU activation function, which is non-differentiable at 0, has become quite popular, e.g. in AlexNet) The first factor is straightforward to evaluate if the neuron is in the output layer, because then  $o_j = y$  and

### >> Section 4

Assumptions The mathematical expression of the loss function must fulfill two conditions in order for it to be possibly used in backpropagation. The first is that it can be written as an average  $E = \frac{1}{n} \sum_x E(x)$  over error functions  $E(x)$ , for  $n$  individual training examples,  $x$ . The reason for this assumption is that the backpropagation algorithm calculates the gradient of the error function for a single training example, which needs to be generalized to the overall error function. The second assumption is that it can be written as a function of the outputs from the neural network.

### >> Section 5

$\frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial o_j} = \frac{\partial E}{\partial y} (t - y) = t - y$   $\{ \displaystyle \frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial o_j} = \frac{\partial E}{\partial y} (t - y) = t - y \}$

### >> Section 6

The loss function is a function that maps values of one or more variables onto a real number intuitively representing some "cost" associated with those values. For backpropagation, the loss function calculates the difference between the network output and its expected output, after a training example has propagated through the network.

# Research Notes on Backpropagation

## Backpropagation

### >> Section 7

In machine learning, backpropagation is a gradient computation method commonly used for training a neural network in computing parameter updates. It is an efficient application of the chain rule to neural networks. Backpropagation efficiently computes the gradient of the loss with respect to the network weights for a single input-output example. It does this by propagating derivatives backward, one layer at a time, from the output layer to the input layer, thereby avoiding redundant chain-rule calculations. Strictly speaking, the term backpropagation refers only to an algorithm for efficiently computing the gradient, not how the gradient is used, but the term is often used loosely to refer to the entire learning algorithm. This includes changing model parameters in the negative direction of the gradient, such as by stochastic gradient descent, or as an intermediate step in a more complicated optimizer, such as Adaptive Moment Estimation. Backpropagation had multiple discoveries and partial discoveries, with a tangled history and terminology (see § History). Some other names for the technique include "reverse mode of automatic differentiation" or "reverse accumulation".

### >> Section 8

Consider the network on a single training case:  $(1, 1, 0)$ . Thus, the input  $x_1$  and  $x_2$  are 1 and 1 respectively and the correct output,  $t$  is 0. Now if the relation is plotted between the network's output  $y$  on the horizontal axis and the error  $E$  on the vertical axis, the result is a parabola. The minimum of the parabola corresponds to the output  $y$  which minimizes the error  $E$ . For a single training case, the minimum also touches the horizontal axis, which means the error will be zero and the network can produce an output  $y$  that exactly matches the target output  $t$ . Therefore, the problem of mapping inputs to outputs can be reduced to an optimization problem of finding a function that will produce the minimal error. However, the output of a neuron depends on the weighted sum of all its inputs: